Looking back

1. For each of the Java programs below, indicate whether it has an error in syntax, an error in program semantics, or no error at all.

   (Assume each code snippet appears inside a valid `main()` method, with no other definitions.)

   a.
   ```
   String s;
   System.out.println(s);
   ```

   b.
   ```
   String s = "unicorn"s";
   ```

   c.
   ```
   int x = (1 + 2)) * 3);
   ```

   d.
   ```
   int x = 10;
   System.out.println(x[20]);
   ```

2. Here is a specification for a very simple language that just lets us do some logic operations.

   (If you don't remember it from your computer architecture class, the NAND operation on two boolean values means "not (X and Y)".)

   ```
   COOLBOOL language

   SYNTAX

   There are three kinds of Expressions:

   *   A single character `T`

   *   A single character `?`

   *   Two Expressions next to each other with no space in between

   A Statement is a single Expression surrounded by square brackets
   `[]`.

   A Program is a sequence of one or more statements.

   SEMANTICS

   *   `T` always evaluates to True.

   *   Evaluating `?` causes a single character to be read in from standard in.
   ```

```
    If that character is `Y`, then the result of the evaluation
    is True; otherwise, the result of the evaluation is False.

*   Two expressions next to each other evaluate to the logical
    NAND of the two values.

A Statement with square brackets evaluates the contained
expression then prints either `true` or `false`.

A Program executes each Statement in order.
```

a. Write a valid COOLBOOL program that will always print `true`.

b. Write a valid COOLBOOL program that will always print `false`.

c. Consider the following COOLBOOL program:

```
[?T][TT?]
```

There are two user inputs (`?`) in this program. Come up with a valid user input (two characters) that would definitely cause this program to print the following:

```
true
true
```

d. Demonstrate that the COOLBOOL language allows **Underspecified Behavior (UB)** by writing a program which follows the spec but could be interpreted in two different ways. Explain how the two possible results could happen.

e. Fix the COOLBOOL spec so that it no longer has UB.

Looking ahead

3. Watch this YouTube video which very enthusiastically gives an introduction to compilation with LLVM and the LLVM IR language which we will be using.

   The whole video is interesting and relevant for us, but you only need to watch **up to 17:30** and can skip the rest where he gets into optimizations and other languages if you don't have time.

   https://www.youtube.com/watch?v=IR_L1xf4PrU

   Answer these questions to check you were paying attention:

   a. Besides C, what is another language that uses LLVM in its compiler?

   b. Is IR generated before, after, or instead of the compiler producing assembly code?

c. What command-line flags do we have to pass to clang so that it outputs LLVM IR code?

d. What does something like `%2` represent in LLVM IR code?